



COMP30026

Models of Computation

Cezary Kaliszyk and William Umboh

Lecture 10

Predicate Logic and Automated Reasoning



Overview

So far

- Unification
- Resolution Step

Today

- Factoring
- Automated Reasoning



Recall: Unification

Input: Two expressions s and t .

Output: If unifiable: mgu.

Algorithm: Complexity?

Recall: Resolution Step

Let P_1 and P_2 be **unifiable** atomic formulas with no variables in common. Let θ be the unifier.

Let C_1 and C_2 be disjunctive clauses.

$$\frac{C_1 \cup \{P_1\} \quad C_2 \cup \{\neg P_2\}}{(C_1 \cup C_2)[\theta]}$$

Recall: Resolution Step

Let P_1 and P_2 be **unifiable** atomic formulas with no variables in common. Let θ be the unifier.

Let C_1 and C_2 be disjunctive clauses.

$$\frac{C_1 \cup \{P_1\} \quad C_2 \cup \{\neg P_2\}}{(C_1 \cup C_2)[\theta]}$$

Remember: Rename variables! (Why?)



Factoring

Consider clause C with unifiable $A_1, A_2 \in C$.

Given mgu θ , add the clause $C[\theta]$.



Factoring

Consider clause C with unifiable $A_1, A_2 \in C$.

Given mgu θ , add the clause $C[\theta]$.

Example

$$\{D(f(y), y), \neg P(x), D(x, y), \neg P(z)\}$$



Factoring

Consider clause C with unifiable $A_1, A_2 \in C$.

Given mgu θ , add the clause $C[\theta]$.

Example

$$\{D(f(y), y), \neg P(x), D(x, y), \neg P(z)\}$$

|

$$\{D(f(y), y), \neg P(f(y)), \neg P(z)\}$$

Factoring

Consider clause C with unifiable $A_1, A_2 \in C$.

Given mgu θ , add the clause $C[\theta]$.

Example

$$\{D(f(y), y), \neg P(x), D(x, y), \neg P(z)\}$$

|

$$\{D(f(y), y), \neg P(f(y)), \neg P(z)\}$$

|

$$\{D(f(y), y), \neg P(f(y))\}$$



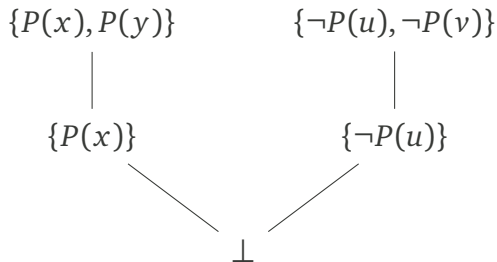
The Need for Factoring

Factoring is sometimes crucial:

$$\{P(x), P(y)\} \qquad \{\neg P(u), \neg P(v)\}$$

The Need for Factoring

Factoring is sometimes crucial:





The Resolution Method

Start with collection Σ of clauses

While $\perp \notin \Sigma$ do

 add to Σ a factor of some $C \in \Sigma$

 or a resolvent of some $C_1, C_2 \in \Sigma$

Question: Does this process always terminate?

- For unsatisfiable inputs?
- For satisfiable inputs?



The Need for Fairness

$\{Q\}$ $\{\neg Q\}$ $\{\neg P(x), P(f(x))\}$

For unsatisfiable inputs terminates...,

The Need for Fairness

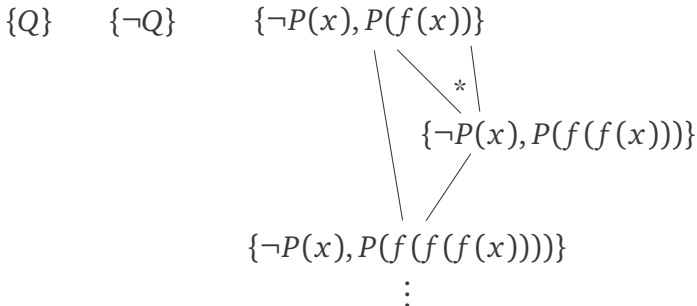
$$\begin{array}{ccc}
 \{Q\} & \{\neg Q\} & \{\neg P(x), P(f(x))\} \\
 & & \swarrow \quad \downarrow \\
 & & \begin{array}{c} * \\ \{ \neg P(x), P(f(f(x))) \} \end{array}
 \end{array}$$

* Here we resolve the clause against **a renamed copy of itself**. Without renaming, unification fails.

For unsatisfiable inputs terminates...,



The Need for Fairness



* Here we resolve the clause against a **renamed** copy of **itself**. Without renaming, unification fails.

For unsatisfiable inputs terminates..., only with sensible search strategy.



The Power of Resolution

Theorem

Σ is unsatisfiable iff the resolution method can add $\perp$ after a finite number of steps.

The Power of Resolution

Theorem

Σ is unsatisfiable iff the resolution method can add $\perp$ after a finite number of steps.

We say that resolution is **sound** and **refutation-complete**.

The Power of Resolution

Theorem

Σ is unsatisfiable iff the resolution method can add $\perp$ after a finite number of steps.

We say that resolution is **sound** and **refutation-complete**.

Not a **decision procedure**: may not terminate on satisfiable formulas.

The Power of Resolution

Theorem

Σ is unsatisfiable iff the resolution method can add $\perp$ after a finite number of steps.

We say that resolution is **sound** and **refutation-complete**.

Not a **decision procedure**: may not terminate on satisfiable formulas.

Indeed, no such procedure can exist for predicate logic.

The Power of Resolution

Theorem

Σ is unsatisfiable iff the resolution method can add $\perp$ after a finite number of steps.

We say that resolution is **sound** and **refutation-complete**.

Not a **decision procedure**: may not terminate on satisfiable formulas.

Indeed, no such procedure can exist for predicate logic.

Validity/unsatisfiability are **semi-decidable** properties.



Horn Clauses and Prolog

A **Horn** clause is a clause with at most one positive literal.

All seven clauses used in the tadpole example were Horn.

A clause such as $\{\neg T(z), \neg D(y), \neg E(y, z), M(z)\}$ can also be thought of as

$$(T(z) \wedge D(y) \wedge E(y, z)) \rightarrow M(z)$$

In the **logic programming** language Prolog, written:

```
miserable(Z) :- tadpole(Z),  
                deepwaterfish(Y),  
                eats(Y, Z).
```



Horn Clauses and Search

For **propositional** Horn clauses, satisfiability can be decided in linear time.

For logic programs, the restriction to Horn clauses simplifies the search problem, but it does not remove it.



Satisfiability of Horn Formulas

1. maintain list of atoms, $\perp$, $\top$ occurring in ϕ



Satisfiability of Horn Formulas

1. maintain list of atoms, $\perp$, $\top$ occurring in ϕ
2. mark $\top$ if it appears in list



Satisfiability of Horn Formulas

1. maintain list of atoms, $\perp$, $\top$ occurring in ϕ
2. mark $\top$ if it appears in list
3. **while** $\exists$ Horn clause $P_1 \wedge \cdots \wedge P_n \implies Q$ in ϕ such that all $P_1, \dots, P_n$ are marked and Q is unmarked

Satisfiability of Horn Formulas

1. maintain list of atoms, $\perp$, $\top$ occurring in ϕ
2. mark $\top$ if it appears in list
3. **while** $\exists$ Horn clause $P_1 \wedge \cdots \wedge P_n \implies Q$ in ϕ such that
all $P_1, \dots, P_n$ are marked and Q is unmarked
mark Q

Satisfiability of Horn Formulas

1. maintain list of atoms, $\perp$, $\top$ occurring in ϕ
2. mark $\top$ if it appears in list
3. **while** $\exists$ Horn clause $P_1 \wedge \cdots \wedge P_n \implies Q$ in ϕ such that
all $P_1, \dots, P_n$ are marked and Q is unmarked
mark Q
4. **if** $\perp$ is marked **then**
return **unsatisfiable**
else
return **satisfiable**

Satisfiability of Horn Formulas

1. maintain list of atoms, $\perp$, $\top$ occurring in ϕ
2. mark $\top$ if it appears in list
3. **while** $\exists$ Horn clause $P_1 \wedge \cdots \wedge P_n \implies Q$ in ϕ such that
all $P_1, \dots, P_n$ are marked and Q is unmarked
mark Q
4. **if** $\perp$ is marked **then**
return **unsatisfiable**
else
return **satisfiable**
satisfying assignment:
$$v(P) = \begin{cases} \top & \text{if } P \text{ is marked} \\ \text{F} & \text{if } P \text{ is unmarked} \end{cases}$$



Equality in Automated Reasoning

Add a new predicate symbol $eq(x, y)$ and axiomatize it

Equality in Automated Reasoning

Add a new predicate symbol $eq(x, y)$ and axiomatize it

Reflexivity, symmetry, transitivity, congruence

$$x = x$$

$$x = y \iff y = x$$

$$x = y \wedge y = z \Rightarrow x = z$$

And for every function symbol f and predicate symbol P :

$$x = y \Rightarrow f(x) = f(y)$$

$$x = y \wedge P(x) \Rightarrow P(y)$$



Equality in Automated Reasoning

Add a new predicate symbol $eq(x, y)$ and axiomatize it

Reflexivity, symmetry, transitivity, congruence

$$x = x$$

$$x = y \iff y = x$$

$$x = y \wedge y = z \Rightarrow x = z$$

And for every function symbol f and predicate symbol P :

$$x = y \Rightarrow f(x) = f(y)$$

$$x = y \wedge P(x) \Rightarrow P(y)$$

Is it efficient?

Automated Reasoning

Field of research since the fifties

- Computer used to reason in a logic
- Traditionally part of artificial intelligence
 - (not machine learning)
- Applications: program verification, mathematical deduction, ...
- Theorem proving logics, precision, automation, ... very varied.
- In this subject we look at automated theorem proving, which is a subpart of this field



Automated Reasoning: Main Question

Given

A set of axioms and a conjecture in a formal language
(e.g. predicate logic)

$$\{A_1, \dots, A_n\}, C$$

Answer

Do the axioms logically imply the conjecture?

$$A_1, \dots, A_n \models C$$

Why is this interesting? (1/2)

Robbins Conjecture

- Open problem in math for many years
- Computer program proved the fact

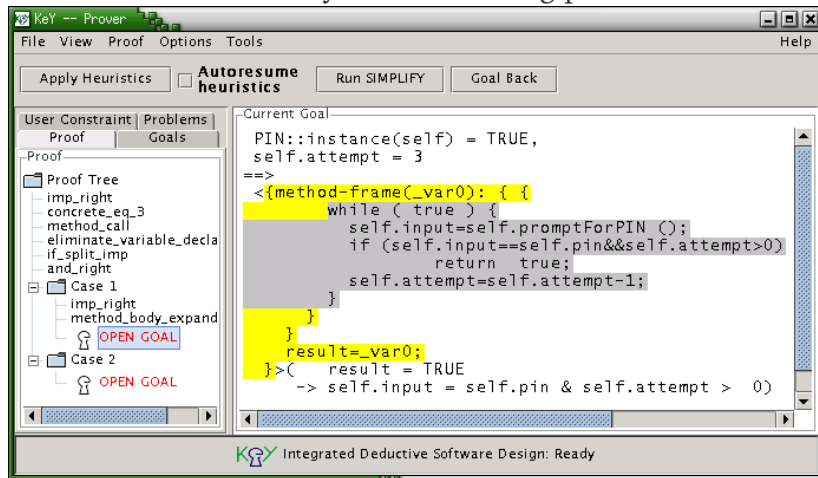
[McCune'1996]

Simple Statement

$$n(n(x) + y) + n(n(x) + n(y)) = x \iff n(n(x + y) + n(x + n(y))) = x$$

Why is this interesting (2/2)

Verification of programs or security protocols
often translated to many small reasoning problems



[Beckert]



Most work in classical predicate logic

Rationale for this

- Compromise between expressivity and automatizability

Expressive

- Many computable problems can be encoded naturally

Automation is possible

- Sound and complete search exists (reasonably efficient)

Translations to and from

- Complex logic, other problems (SQL, graphs, scheduling, ontology...)